Building CoworkEnv: Practical Lessons for Training Cowork Agents with Limited Resources

CoworkEnv Team

Companion notes to the CoworkEnv paper

Why we are sharing these notes

CoworkEnv turns workflow requirements into executable environments for training AI coworkers. PocketWorker is the model family trained with those resources. Our paper describes task construction, teacher demonstrations, supervised fine-tuning, and reinforcement-learning experiments [1]. This companion shares the practical decisions behind that work: how to make an environment trustworthy, find useful learning signals, and spend a limited research budget on experiments that answer clear questions.

The development record contains small interface failures with large consequences. A plausible tool name pointed to the wrong endpoint. A reward existed under a field that its consumer did not read. A training run made real updates but left no recoverable checkpoint. Each case suggests a concrete check, and together they suggest an order of work: establish a working interaction, verify what reaches the loss, then expand the experiment.

These notes cover historical development and subsequent exploration, including process-feedback calibration and a Tinker experiment with a model in the 30B range. The paper identifies the configurations and benchmark results of the released model series. Here, the emphasis is on mechanisms, decisions, and reusable practices. Suggested workflows are recommendations distilled from that experience.

A route through the lessons

- **Part I - Make tasks and training work:** align task contracts, validate grading, inspect actual supervision, and preserve the interaction protocol.
- **Part II - Make experiments affordable:** start small, measure usable feedback, choose compute deliberately, and preserve recoverable progress.
- **Part III - Turn observations into research:** separate optimization mechanisms, calibrate process feedback, and turn surprises into focused experiments.

> **The organizing principle:** Use the smallest experiment that can resolve the current uncertainty. Increase scale after the result tells you what the next experiment should test.

I / MAKE TASKS AND TRAINING WORK

01 Build one executable contract

A cowork task joins a request, initial records, tool interfaces, and grading checks. Those pieces must agree about entities, actions, and the intended final state. A support workflow might connect a ticket to a customer, retrieve an account manager's address, and send the correct escalation. Individually plausible components can still form a broken workflow.

A concrete repair. Five generated tasks pointed a contacts operation at `/contacts/send`, while the service expected `/contacts/send_message`. Another task requested three to-do items, but its grader required only two successful creation calls. Correcting the endpoint and the count repaired different parts of the same contract: what the agent could do and what the evaluator would accept.

Validate that contract in three layers. First, check the specification: registered services, fixture fields, entity links, tool references, check weights, and required versus prohibited actions. Inspect the packaged instruction as well. A reference answer stored separately during generation can become visible if a later registration step copies it into the request. Scanning must distinguish legitimate starting information from information the task asks the agent to discover.

Second, exercise the grader on known states or traces. CoworkEnv's grading-consistency gate checks a synthetic complete record, an empty record, and a complete record with a forbidden action. These cases establish expected scoring behavior for the declared predicates. Third, execute a known valid route through the real services. This catches unreachable endpoints, missing fields, and state transitions absent from a synthetic trace.

Keep a few representative workflows as integration fixtures. Include cross-service identity resolution, a state-changing action, a time-dependent condition, and a constrained action. When a shared service or grader template changes, replay those workflows before generating a larger collection. The useful unit of progress is an environment that can support a measured interaction.

> **Practice:** Preserve separate checks for specification consistency, grader behavior, and live execution. Run the inexpensive checks first, then verify the packaged task through its actual tools.

I / MAKE TASKS AND TRAINING WORK

02 Make grading follow observable effects

A well-formed score is useful only when its predicates correspond to the behavior of interest. A successful dispatch, a recorded state change, and a correct final artifact establish different things. Choose the evidence according to the task: a sent-message receipt can establish delivery, while an explanation may need semantic assessment.

The project investigations found audit-key mismatches and aliases that lost their logical action identity when routed through shared endpoints. If a grader expects one audit collection while the service writes another, a completed action can receive no credit. If two tools collapse to one reverse-mapped name, a forbidden-action check may examine the wrong identity. Trace the complete path from arguments to service mutation, audit record, and grading input. Pass environmental assumptions, including a fixed reference date, through that path too.

Build contrast cases that distinguish meaningful behaviors: no action, correct partial progress, complete work, the wrong entity, an incorrect final artifact, repeated actions, and a prohibited operation. Counting three successful calls may still accept three copies of the same item. Test the requested result as directly as the environment permits.

Component scores help explain edge cases. In one investigated formula, an empty dispatch receives maximal robustness. With safety equal to one, completion equal to zero, and a robustness weight of 0.2, its aggregate is 0.2. CoworkEnv's empty-record consistency check targets zero completion. Keeping those components visible avoids confusing an expected formula outcome with a failed test.

A proxy language-model judge can test scoring composition and error handling. Representative outputs assessed by the real judge are needed to examine semantic discrimination. Record assessment failures separately from scored task failures, and retain evidence that explains each score. The paper's safety example uses action-based checks; those checks should be interpreted according to the actions they observe.

> **Practice:** Design negative cases as carefully as positive ones. Preserve action identity, inspect final effects, and keep score components available when training consumes an aggregate.

I / MAKE TASKS AND TRAINING WORK

03 Inspect the supervision that reaches the model

A trajectory passes through conversion, rendering, tokenization, truncation, and collation before it becomes a training example. Every stage can change what the model sees or predicts. A field surviving in JSON does not establish that the selected chat template renders it.

A historical failure. A task-context loader used the wrong module path and caught exceptions by returning no system context or tools. A repair corrected the lookup and removed the silent fallback. Another change moved reasoning into a representation consumed by the selected template. These changes affected the actual supervision. The repository later contained converter variants with different behavior, making the executed version important.

Inspect a small set of fully rendered examples with token-level labels. For the assistant-target objective in the paper, user and tool messages provide context, while selected assistant responses and calls receive supervision. Verify the final masks after packing and truncation. An early assignment of input tokens to labels does not establish the eventual mask; framework settings distinguish full-sequence and assistant-only training [6].

Preserve each action's structure. Text and tool calls belonging to one assistant turn should retain that boundary, followed by observations linked through call identifiers. Distinct turns separated by tool observations remain distinct. If a result is missing, emitting an empty placeholder preserves an identifier match but does not recover the evidence needed by the next action.

Selection also has distinct stages. In the paper's mixed-pool path, reward filtering occurs at export; conversion then checks structure and length. The inspected converter clips long tool observations to a 1,000-character prefix and rejects examples above a 40,000-character budget. The trainer subsequently enforces its token limit. These operations can remove different information, so inspect the final example rather than assuming that a retained row is intact.

> **Practice:** Keep rendered, tokenized, and labeled regression examples alongside the converter. Check selection, representation, supervision scope, and retained evidence separately.

I / MAKE TASKS AND TRAINING WORK

04 Keep the interaction protocol consistent

An agent's behavior depends on the weights and the protocol around them. The teacher, training renderer, model server, tool executor, and evaluator must agree about how actions and observations are represented. An output that resembles a tool call still needs to be parsed and dispatched.

One reported serving path received Qwen-style text inside tool-call tags while its consumer expected a structured API field. The bridge needed to render schemas into context, parse generated calls, execute them, and reconstruct call history for the next model turn. Test names, argument types, call identifiers, tool errors, and multi-call responses through the whole loop. Parser configuration is an explicit part of serving systems such as SGLang [7].

A useful fixture is one complete conversation: an initial request, a tool call, its observation, and a follow-up action. Run it through collection, SFT rendering, rollout, serving, and evaluation where those interfaces apply. Compare the intended content at each boundary. Check the effective system prompt, tool schemas, stop conditions, history policy, and interaction budget. A trace-reader failure should remain visible as a parsing error.

Sampling settings deserve direct inspection. A generation path can accept a temperature argument yet resolve to greedy decoding. That matters when investigating diversity within RL groups. Log the resolved settings and the model assets loaded by the running process, including tokenizer, processor, and template.

Representation consistency and numerical sampling/training consistency are separate checks. Missing observations or schemas alter the prefix. Numerical mismatch concerns probabilities, precision, stale weights, or other engine differences for the intended prefix. Diagnose the relevant layer before changing the optimization objective. An alignment correction to log probabilities cannot restore information removed from context.

> **Practice:** Make a complete tool interaction part of every deployment check. Preserve the effective prompts, schemas, decoding settings, and trace format with the experiment.

II / MAKE EXPERIMENTS AFFORDABLE

05 Start with the smallest decisive experiment

Limited compute makes the order of experiments especially important. A full training run changes many things at once and can take hours to expose an interface error. We recommend a progression in which each stage answers one question before the next stage consumes more resources.

First, replay without a policy. Use fixed actions and known states to test tool routing, fixtures, grading, serialization, and failure handling. Many contract errors can be found on CPU. Establish a valid solution route separately, through scripted execution or a capable teacher.

Second, exercise a small policy. Select a handful of representative tasks and a smaller model with a compatible interaction format. Check actual calls, returned observations, continuation, and termination. Prefer tasks that expose different interfaces over many copies of an easy task. A small policy's failure to solve a difficult workflow should trigger diagnosis; it cannot establish that the workflow is impossible.

Third, perform a small learning test. Inspect labels or advantages, unmasked action tokens, and parameter updates on a deliberately informative batch. For SFT, a tiny overfit test can check whether the intended targets can be learned. For RL, exercise known reward contrasts and then a short live loop. Save and reload the resulting state, and confirm that generation still works. These are integration tests, not benchmark measurements.

Fourth, test the expensive boundary early. Include a long history, a large observation, a service failure, and an interrupted run. Use the intended backend before scaling to many workers. Shorten the number of tasks or updates first; reducing the interaction horizon below the steps required by the task can change the problem itself.

Finally, expand one dimension at a time. Increase task variety, model size, context length, or concurrency after the preceding check passes. Repeat a small acceptance suite on the target model because tokenization, architecture, and memory behavior can change. Reserve independent tasks for capability evaluation.

This progression complements the paper's 4B and 8B experiments; it is a proposed development workflow, not a claim about the historical ordering of those runs. Its purpose is to discover failures while their cost is still small.

> **Practice:** Before launching a run, write the uncertainty it resolves and the smallest observable result that would resolve it. Keep debugging fixtures separate from capability evaluations.

II / MAKE EXPERIMENTS AFFORDABLE

06 Budget for usable feedback

Rollout throughput measures activity. Learning also requires valid measurements and useful distinctions between behaviors. Track how much of the sampling budget reaches the intended objective.

An investigated interface mismatch illustrates the first requirement. The rollout producer wrote `reward`, while its consumer read `score`. The missing lookup became an invalid sample. Generation continued, but the reward-driven policy signal disappeared. Validate a canonical reward schema, including required components, before group processing. A valid score of zero remains data; unavailable grading needs a status and reason.

Group-relative methods add a second requirement. Equal rewards yield zero-centered advantages. A standard-deviation floor limits amplification of small differences but creates no contrast when all rewards are equal [2]. A zero-variance group can be valid while supplying no reward-driven policy gradient. This is distinct from infrastructure rejection and from auxiliary losses or optimizer-state effects.

What historical accounting revealed. A review of two development logs found 561 group summaries in each. One classified 48 as accepted, 342 as zero-variance, and 171 as infrastructure-invalid. The other classified 372, 54, and 135, respectively. These historical categories describe different configurations; the counts illustrate how equal iteration budgets can hide very different feedback availability. Current logging conventions must be interpreted on their own terms.

Task-selection statistics answer another question. The paper's safety example was retained after three teacher completion scores of 0.8. Intermediate average completion can justify collection without guaranteeing within-group reward variation. Measure the student's sampled trajectories under the actual training settings before inferring useful RL contrast.

Track attempted episodes, valid scores, informative groups, failure reasons, retries, action tokens, and elapsed time. Inspect these by task family. A coarse grader, insufficient horizon, deterministic decoding, uniform success, and uniform failure can all produce constant rewards. Resource and algorithm changes should follow that diagnosis.

> **Practice:** Monitor measured feedback and useful variation alongside speed. Keep unavailable assessments, valid zero scores, and valid zero-variance groups distinguishable throughout the pipeline.

II / MAKE EXPERIMENTS AFFORDABLE

07 Choose compute around the experiment

Research resources include GPU capacity, CPU environments, external judges, and the time spent maintaining them. A training backend is a practical choice about which parts of the system the team needs to control directly.

An exploratory route. During development, we used Tinker to fine-tune a model in the 30B range. This experiment was separate from the released PocketWorker checkpoints. It provides a concrete example of exploring a larger model through managed training infrastructure while keeping the formal release tied to the training paths documented in the paper.

Tinker's API provides sampling, gradient computation, optimizer steps, and state management for LoRA training. Data preparation and reward functions remain under the user's control [8]. Its Harbor recipe also exposes a sandbox interface for custom backends [9]. For a project centered on environment construction and supervision, this is a useful option to evaluate alongside self-managed training. Work requiring a new critic architecture or deeper backend changes needs an explicit compatibility check; custom loss support has a defined interface [10].

Choose between these paths using the actual experiment. Compare setup time, debugging effort, supported models and objectives, context limits, checkpoint access, and the cost of sampling and training. Avoid inferring savings from model parameter count alone. For API-backed work, record input, output, and training tokens as well as judge calls. For local work, include utilization and waiting time in addition to GPU hours.

The environment can become the bottleneck in either setup. Lease ports and sandbox instances over the full episode lifecycle. A queue can coordinate a shared pool, but its scope must cover all competing workers. Verify service identity after startup. Bound grader retries and preserve their history so recovery does not silently select the best score from repeated attempts.

Our practical preference is to make infrastructure decisions serve the current research question. A managed backend may reduce the work required to test a data or reward idea; a controlled local stack may be necessary to study the training mechanism itself.

> **Practice:** Compare time to a trustworthy experiment, required control, and total operating cost. Plan model compute, environment capacity, and judge availability together.

II / MAKE EXPERIMENTS AFFORDABLE

08 Preserve progress and test the artifact

A long run should produce recoverable state before it reaches its final update. Checkpoint scheduling, tracking, export, and serving are part of the experiment's execution plan.

A costly failure mode. An exploratory PPO run reached actor step 122 before sandbox command-output decoding failed and retries were exhausted. Its checkpoint interval was one million updates, larger than its 187-update plan. The run had made real training progress but saved no training checkpoint. A subsequent repair addressed decoding and changed the save interval to 25. The case makes recovery policy concrete without relying on the replacement run's eventual outcome.

Test saving and resumption during the small learning test. Preserve optimizer and critic state when the experiment requires them, and record whether a restart actually resumes or begins again from the initializer. Keep local progress logs even when an external tracker is unavailable. Confirm the first recoverable checkpoint early in the run.

Long-horizon resource planning needs the same attention. Distinguish per-turn generation, retained tool payload, complete context, interaction count, and loss normalization. The paper's 1,536-token per-turn generation limit is not a complete-trajectory budget. Profile rollout, forward/backward computation, and saving with representative long histories. Observation tokens consume memory and compute even when their direct loss mask is zero.

Finally, exercise the exported model through the intended serving stack. A health endpoint cannot establish usable generation. Check ordinary responses, a structured tool call, a multi-turn continuation, and supported modalities. Preserve the source weights or adapter, export configuration, tokenizer, processor, and template. Compare behavior before and after merge or conversion, and version any compatibility edits.

Bind the resulting evaluation to the artifact actually loaded by the server. Paths and aliases help humans navigate; the saved transformation record and content identities establish what was tested.

> **Practice:** Save early, test resumption, and verify real inference after export. Treat updates, recoverable checkpoints, and deployable artifacts as separate milestones.

III / TURN OBSERVATIONS INTO RESEARCH

09 Describe the learning mechanism precisely

Agent RL configurations differ along several axes: reward construction, advantage estimation, the actions receiving each signal, and policy-loss reduction. Naming these choices makes a comparison interpretable and helps locate implementation errors.

Consider an illustrative reward with safety fixed to one: `0.8 * completion + 0.2 * robustness`. Component pairs `(0.25, 1.0)` and `(0.50, 0.0)` both yield 0.40. Their scalar rewards supply no contrast, even though their behaviors differ. GDPO normalizes components separately before combining them and applying batch normalization [4]. This can preserve distinctions erased by scalar aggregation, although it does not guarantee a nonzero signal for every pattern.

Dr.GRPO changes advantage and loss normalization [3]. The paper's implementation removes division by group reward standard deviation and uses a fixed loss normalizer. GiGPO adds state-group advantages to episode-level advantages [5]. Its usefulness depends on meaningful comparisons at intermediate states; an anchor shared only at initialization does not establish local feedback later in a workflow. GSPO instead changes the sequence importance ratio and clipping operation.

Method identity should be observed in the executed path. A historical log with a GDPO-looking filename recorded a GRPO estimator. DR-Rubric names a reward configuration, not Dr.GRPO. Conversely, a helper reporting GRPO-style group preprocessing does not establish that the actor's sequence-level objective was disabled. Record both preprocessing and the actual update path, with the executed revision.

The paper compares complete configurations. Its GDPO comparison also changes batch size, and its Dr.GRPO comparison changes two normalizations. Preserve that scope when interpreting results. Raw policy-loss magnitudes with different reductions are not a shared quality scale; a small mean loss can also coexist with nonzero gradients.

For a narrower next experiment, hold outcome feedback, tasks, initialization, and budgets fixed while changing the estimator or assignment rule. Inspect which decisions receive a distinct signal, then evaluate the resulting policy. Per-turn scores summed into one broadcast advantage still provide episode-level assignment.

> **Practice:** Record reward, estimator, temporal placement, and loss reduction independently. Attribute improvements to the tested configuration until a controlled comparison isolates a narrower mechanism.

III / TURN OBSERVATIONS INTO RESEARCH

10 Calibrate process feedback before using it

Process feedback asks an evaluator to assess partial work throughout a trajectory. A reliable terminal rubric is a starting point, but intermediate feedback also needs to distinguish new progress, repeated evidence, uncertainty, and loss of an earlier achievement.

An exploratory calibration. A frozen phone-and-triage batch contained 39 synthetic-prefix cases. Its 72 new model requests all returned, while 70 responses passed protocol validation. Full case-level review accepted 31 response pairs and rejected eight. The cases also included a reused historical reviewer response and native safety checks, so requests and cases have different denominators. These are calibration results for selected prefixes, separate from the paper's benchmark evaluations.

Seven rejected pairs involved unsupported numeric judgments about media whose binding to the task instance was unresolved. A matching path or available caption did not establish that the content belonged to the required task. Two production responses retained an unknown predicate but still produced a numeric aggregate inconsistent with their contract. Those failures overlapped with the rejected pairs.

Another response gave a supported current score but invented an earlier completed report that had supposedly been destroyed. This is a useful distinction: correctness of the current scalar does not establish correctness of the claimed transition. A negative increment needs evidence that a previously achieved condition was actually invalidated.

Define state assessment and reward increment separately. If reward pays for newly completed work, repeated observations should not repeatedly earn the same credit. Preserve unknown outcomes when evidence is insufficient. Include incorrect artifacts, missing evidence, repeated actions, and verified regressions in calibration. Inspect real agent trajectories after synthetic cases pass, because natural interactions introduce different ambiguity and failure patterns.

Keep expected predicates and input provenance fixed before model review. Record the state assessments, increments, and supporting observations. This makes disagreement informative and helps determine whether a dense reward stream follows actual progress.

> **Practice:** Validate evidence, current state, and temporal change separately. A successful API call or plausible score is only one part of accepting process supervision.

III / TURN OBSERVATIONS INTO RESEARCH

11 Turn surprises into better experiments

The recurring lesson across development is a change in what deserves attention. A completed job, a generated task, or an improving curve becomes useful research evidence when its connection to behavior is understood. The practical response is to make the next experiment more specific.

When behavior shifts, inspect before explaining. Reports described an intermediate model narrating actions where tool calls were expected. Data mixture, rendering, and serving can all affect that symptom. First verify the interaction path, then compare checkpoints on a stable development set. Track tool use, clarification, and artifact production separately. A retention pool is an intervention to test; its effect should be separated from changes in ordering, learning rate, and training duration.

When more data helps, ask which additional experience helped. The paper's stage-wise 20%, 50%, and full-data experiment supports the utility of more supervision in those settings. It motivates finer questions about coverage, repetition, and trajectory quality. Count examples, supervised tokens, actions, and consumed samples separately. A larger input file does not necessarily imply proportionally greater exposure in a truncated run.

When a larger model becomes accessible, keep the question fixed. A larger model can help explore capacity limits or provide a capable reference, while a smaller model can accelerate interface checks. Before changing scale, decide whether the experiment tests learning, task feasibility, or system behavior. Repeat essential acceptance checks on the target model and avoid treating a small-model result as a universal prediction.

When a run fails, preserve what it taught. Keep the configuration, symptom, evidence, intervention, and remaining uncertainty in a short decision record. A useful account explains which decision changed: adding a grader contrast, fixing the renderer, changing the checkpoint schedule, or choosing a different compute route. It need not turn every repair into a performance claim.

For future work, we would prioritize comparisons that connect feedback quality to its allocation: fixed outcome feedback with alternative estimators, then a common calibrated process-feedback condition. Measure environment interactions, judge requests, wall-clock time, and capability together. This reveals whether a method's additional complexity earns its cost.

> **Practice:** End each investigation with a narrower question and an experiment that can answer it. Keep the decision record compact enough to guide the next person who encounters the same symptom.

QUICK REFERENCE

A short operating guide

Before a long run

- State the question, the comparison, and the condition for stopping or expanding.
- Replay a known valid workflow and representative grader contrasts.
- Inspect a rendered example, its final labels, and one complete tool interaction.
- Exercise reward fields, valid groups, advantages, and updates on a diagnostic batch.
- Test long-history behavior, checkpoint saving, reload, and interruption recovery.

While the run is active

- Track valid scores, informative groups, failures, retries, and consumed tokens.
- Inspect behavior by task family and record changes to effective configuration.
- Verify recoverable checkpoints and retain local logs alongside external tracking.

Before sharing results

- Test the exported artifact through actual generation and tool execution.
- Bind the result to weights, renderer, harness, task population, and trial rule.
- Aggregate unrounded scores and keep missing results distinct from valid zeros.
- Identify whether a case concerns a released experiment, development history, or subsequent exploration.

READING

References and further reading

Project cases are supported by a separate internal evidence index. The sources below provide the paper, method definitions, and public implementation guidance. Tinker documentation was checked on 20 October 2026; supported models and service details can change.

1. Xie et al. (2026). **CoworkEnv: Synthesizing Training Environments for Long-Horizon Cowork Agents.** Paper, 16 October version.

2. Shao et al. (2024). **DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.** arXiv:2402.03300. GRPO and group-relative learning signals.

3. Liu et al. (2025). **Understanding R1-Zero-Like Training: A Critical Perspective.** arXiv:2503.20783. Dr.GRPO and normalization choices.

4. Liu et al. (2026). **GDPO: Group reward-Decoupled Normalization Policy Optimization for Multi-reward RL Optimization.** arXiv:2601.05242. Component-wise reward normalization.

5. Feng et al. (2025). **Group-in-Group Policy Optimization for LLM Agent Training.** arXiv:2505.10978. Episode- and state-grouped advantages.

6. Hugging Face. **TRL SFT Trainer, v0.23.0.** Documentation. Loss scope and assistant-message masking.

7. SGLang. **Tool Parser.** Documentation source. Serving configuration for structured tool use.

8. Thinking Machines Lab. **Tinker Quickstart.** Documentation. Training, sampling, and local reward computation.

9. Thinking Machines Lab. **Harbor RL.** Recipe and sandbox interface. Harbor-formatted tasks and custom environment backends.

10. Thinking Machines Lab. **Custom Loss Functions.** Documentation. The interface for losses defined over model log probabilities.